Splines and wavelets in detail

Wavelets and splines are very effective tools once you understand these bases in detail.

1 Piecewise nature of B-splines

This is the book on splines: `https://link.springer.com/book/9780387953663`

A B-spline basis function is a polynomial on each subinterval determined by the knot sequence. Suppose the knots are

$$\xi_0 \le \xi_1 \le \cdots \le \xi_m.$$

These knots partition the domain into intervals

$$[\xi_0, \xi_1),\ [\xi_1, \xi_2),\ \ldots,\ [\xi_{m-1}, \xi_m].$$

A B-spline of degree p is defined recursively through the formula. For degree 0,

$$B_{i,0}(x) = \begin{cases} 1, & \xi_i \le x < \xi_{i+1}, \\ 0, & \text{otherwise}, \end{cases}$$

so $B_{i,0}(x)$ is piecewise constant.

For $p \ge 1$,

$$B_{i,p}(x) = \frac{x - \xi_i}{\xi_{i+p} - \xi_i} B_{i,p-1}(x) + \frac{\xi_{i+p+1} - x}{\xi_{i+p+1} - \xi_{i+1}} B_{i+1,p-1}(x),$$

with the convention that terms with zero denominator are taken as zero.

This recursion shows that if $B_{i,p-1}(x)$ and $B_{i+1,p-1}(x)$ are piecewise polynomials of degree $p-1$, then $B_{i,p}(x)$ is a piecewise polynomial of degree p. Hence, by induction, $B_{i,p}(x)$ is a polynomial of degree p on each knot interval.

For example, a cubic B-spline ($p = 3$) is cubic on each interval

$$[\xi_j, \xi_{j+1}),$$

but the cubic expression may differ from one interval to the next. Thus, a cubic B-spline is not a single global cubic polynomial; rather, it is a collection of cubic pieces joined smoothly at the knots.

Two important properties follow:

- **Local support:** $B_{i,p}(x)$ is nonzero only on

  $$[\xi_i, \xi_{i+p+1}),$$

  so each basis function affects only a local region of the domain.

- **Smoothness:** if the knots are simple, then a degree-p B-spline is $p-1$ times continuously differentiable at interior knots. In particular, cubic B-splines are twice continuously differentiable.

Therefore, B-splines provide a flexible basis for function approximation by representing a function as

$$f(x) = \sum_i \beta_i B_{i,p}(x),$$

where each basis function is piecewise polynomial, locally supported, and smoothly connected across knot boundaries.

```
# ------------------------------------------
# Piecewise-ness of B-splines: simple example
# ------------------------------------------

# We use the splines package from base R
library(splines)

# A cubic B-spline basis on [0, 1]
# Internal knots split the domain into subintervals
x <- seq(0, 1, length.out = 500)
internal_knots <- c(0.25, 0.5, 0.75)

# Construct cubic B-spline basis (degree = 3)
B <- bs(
  x,
  knots = internal_knots,
  degree = 3,
  intercept = TRUE,
  Boundary.knots = c(0, 1)
)

# Inspect dimension: each column is one basis function
dim(B)

# Plot all basis functions
matplot(
  x, B, type = "l", lty = 1, lwd = 2,
  xlab = "x", ylab = "Basis value",
  main = "Cubic B-spline basis functions"
)
abline(v = c(0, internal_knots, 1), lty = 2, col = "gray")

# ------------------------------------------------
# Show piecewise polynomial behavior on each interval
# ------------------------------------------------

# Pick one basis function
j <- 3
bj <- B[, j]

plot(
```

```
  x, bj, type = "l", lwd = 2,
  xlab = "x", ylab = paste0("B_", j, "(x)"),
  main = paste("Basis function", j, ": piecewise cubic")
)
abline(v = c(0, internal_knots, 1), lty = 2, col = "gray")

# Fit separate cubic polynomials on each knot interval
intervals <- list(
  c(0.00, 0.25),
  c(0.25, 0.50),
  c(0.50, 0.75),
  c(0.75, 1.00)
)

piece_fits <- vector("list", length(intervals))

for (k in seq_along(intervals)) {
  a <- intervals[[k]][1]
  b <- intervals[[k]][2]
  idx <- which(x >= a & x <= b)

  # cubic polynomial fit on this interval
  dat <- data.frame(x = x[idx], y = bj[idx])
  fit <- lm(y ~ poly(x, 3, raw = TRUE), data = dat)
  piece_fits[[k]] <- list(interval = c(a, b), fit = fit, idx = idx)

  # overlay fitted cubic
  lines(x[idx], predict(fit), col = 2, lwd = 2, lty = 2)
}

legend(
  "topright",
  legend = c("B-spline basis", "Cubic fit on each interval"),
  col = c(1, 2), lty = c(1, 2), lwd = 2, bty = "n"
)

# ------------------------------------------------
# Print polynomial coefficients on each interval
# ------------------------------------------------

for (k in seq_along(piece_fits)) {
  cat("\nInterval:", paste(piece_fits[[k]]$interval, collapse = " to "), "\n")
  print(coef(piece_fits[[k]]$fit))
}

# ------------------------------------------------
# Partition of unity check
# For B-splines with intercept=TRUE, rows sum to 1 inside the boundary range
```

```
# ------------------------------------------------

row_sums <- rowSums(B)
range(row_sums)

plot(
  x, row_sums, type = "l", lwd = 2,
  xlab = "x", ylab = "Sum of basis functions",
  main = "Row sums of B-spline basis"
)
abline(h = 1, lty = 2, col = 2)

# ------------------------------------------------
# Local support: each basis function is nonzero only on a few intervals
# ------------------------------------------------

nonzero_ranges <- apply(B, 2, function(z) {
  idx <- which(abs(z) > 1e-10)
  c(min(x[idx]), max(x[idx]))
})

t(nonzero_ranges)
```

2 Piecewise and localized nature of wavelets

Wavelets form a family of basis functions that are localized in both position and scale. Unlike global polynomial bases, wavelets are designed to capture local features of a function, such as jumps, edges, and transient behavior.

Scaling and wavelet functions. A wavelet system is built from two fundamental functions:

- the *scaling function* $\phi(x)$, which captures coarse or low-frequency structure, and
- the *mother wavelet* $\psi(x)$, which captures local detail.

The simplest example is the Haar system. Its scaling function is

$$\phi(x) = \begin{cases} 1, & 0 \le x < 1, \\ 0, & \text{otherwise}, \end{cases}$$

and its mother wavelet is

$$\psi(x) = \begin{cases} 1, & 0 \le x < \frac{1}{2}, \\ -1, & \frac{1}{2} \le x < 1, \\ 0, & \text{otherwise}. \end{cases}$$

Both functions are *piecewise constant*. In particular, the Haar wavelet is constant on each subinterval but changes value abruptly at the midpoint. This is the simplest illustration of the piecewise nature of wavelets.

Dilations and translations. From the mother wavelet ψ, one constructs a family of wavelet basis functions by dilation and translation:

$$\psi_{j,k}(x) = 2^{j/2}\psi(2^j x - k), \qquad j \in \mathbb{Z}, \quad k \in \mathbb{Z}.$$

Here:

- j controls the *scale* or resolution,
- k controls the *location*.

As j increases, the wavelet becomes more compressed, so it captures finer local structure. Since ψ has compact support, each $\psi_{j,k}$ is nonzero only on a small interval, which gives wavelets their local character.

Piecewise structure. For the Haar system, each $\psi_{j,k}(x)$ remains piecewise constant. More generally, many wavelet families, such as Daubechies wavelets, are not piecewise polynomial in exactly the same simple way as splines, but they are still constructed from localized building blocks with compact support and multiscale structure. Thus, wavelets may be viewed as localized basis functions defined over small regions of the domain, with each basis element contributing only locally.

Multiresolution representation. A function $f(x)$ can be expanded as

$$f(x) = \sum_k c_{J_0,k}\phi_{J_0,k}(x) + \sum_{j \geq J_0} \sum_k d_{j,k}\psi_{j,k}(x),$$

where

$$\phi_{j,k}(x) = 2^{j/2}\phi(2^j x - k).$$

The coefficients $c_{J_0,k}$ describe the coarse-scale approximation of the function, while the coefficients $d_{j,k}$ describe details at different resolutions.

Why wavelets are useful. Because wavelets are localized in both space and scale, they are especially effective for representing functions with spatially inhomogeneous features. Smooth parts of the signal can often be represented with few coefficients, while abrupt local changes are captured by a small number of detail coefficients near the relevant locations.

```
# ------------------------------------------
# Wavelets: localized, piecewise basis functions
# ------------------------------------------

# Install if needed:
# install.packages("wavelets")

library(wavelets)

# ---------------------------------------------------
# 1. Haar scaling and wavelet functions explicitly
# ---------------------------------------------------
```

```
x <- seq(0, 1, length.out = 1000)

# Haar scaling function phi(x):
# phi(x) = 1 on [0,1), 0 otherwise
phi <- function(x) {
  ifelse(x >= 0 & x < 1, 1, 0)
}

# Haar mother wavelet psi(x):
# psi(x) = 1 on [0,1/2), -1 on [1/2,1), 0 otherwise
psi <- function(x) {
  ifelse(x >= 0 & x < 0.5, 1,
         ifelse(x >= 0.5 & x < 1, -1, 0))
}

par(mfrow = c(1, 2))
plot(x, phi(x), type = "l", lwd = 2,
     xlab = "x", ylab = expression(phi(x)),
     main = "Haar scaling function")
abline(v = c(0, 1), lty = 2, col = "gray")

plot(x, psi(x), type = "l", lwd = 2,
     xlab = "x", ylab = expression(psi(x)),
     main = "Haar mother wavelet")
abline(v = c(0, 0.5, 1), lty = 2, col = "gray")

# ----------------------------------------------------
# 2. Dilated and translated Haar wavelets
#    psi_{j,k}(x) = 2^{j/2} psi(2^j x - k)
# ----------------------------------------------------

haar_wavelet <- function(x, j, k) {
  2^(j / 2) * psi(2^j * x - k)
}

par(mfrow = c(2, 2))
plot(x, haar_wavelet(x, 0, 0), type = "l", lwd = 2,
     xlab = "x", ylab = "", main = expression(psi[0,0](x)))
abline(v = c(0, 0.5, 1), lty = 2, col = "gray")

plot(x, haar_wavelet(x, 1, 0), type = "l", lwd = 2,
     xlab = "x", ylab = "", main = expression(psi[1,0](x)))
abline(v = c(0, 0.25, 0.5), lty = 2, col = "gray")

plot(x, haar_wavelet(x, 1, 1), type = "l", lwd = 2,
     xlab = "x", ylab = "", main = expression(psi[1,1](x)))
abline(v = c(0.5, 0.75, 1), lty = 2, col = "gray")
```

```
plot(x, haar_wavelet(x, 2, 2), type = "l", lwd = 2,
     xlab = "x", ylab = "", main = expression(psi[2,2](x)))
abline(v = c(0.5, 0.625, 0.75), lty = 2, col = "gray")

# ------------------------------------------------
# 3. Show piecewise-constant structure numerically
# ------------------------------------------------

w <- haar_wavelet(x, 2, 1)

plot(x, w, type = "l", lwd = 2,
     xlab = "x", ylab = "value",
     main = "A Haar wavelet is piecewise constant")
abline(v = c(0.25, 0.3125, 0.375), lty = 2, col = "gray")

# Unique values (approximately) taken by the wavelet
sort(unique(round(w, 6)))

# ------------------------------------------------
# 4. Discrete wavelet transform of a signal
# ------------------------------------------------

# A simple signal with local features
n <- 128
t <- seq(0, 1, length.out = n)
f <- sin(2 * pi * t) + 0.8 * (t > 0.35) - 0.6 * (t > 0.7)

par(mfrow = c(1, 1))
plot(t, f, type = "l", lwd = 2,
     xlab = "t", ylab = "f(t)",
     main = "Signal with local changes")

# Haar DWT
dwt_fit <- dwt(f, filter = "haar", n.levels = 4, boundary = "periodic")

# Inspect coefficients
names(dwt_fit@W)   # detail coefficients
names(dwt_fit@V)   # scaling coefficients

# Plot wavelet coefficients by level
par(mfrow = c(2, 2))
for (lev in 1:4) {
  coef <- dwt_fit@W[[lev]]
  plot(coef, type = "h", lwd = 2,
       xlab = "Index", ylab = "Coefficient",
       main = paste("Detail coefficients, level", lev))
}
```

```
# ------------------------------------------------
# 5. Reconstruction from coarse + detail pieces
# ------------------------------------------------

# Full reconstruction
rec_full <- idwt(dwt_fit)

plot(t, f, type = "l", lwd = 2,
     xlab = "t", ylab = "Signal",
     main = "Original and reconstructed signal")
lines(t, rec_full, lty = 2, lwd = 2)

legend("topright",
       legend = c("Original", "Reconstructed"),
       lty = c(1, 2), lwd = 2, bty = "n")

# ------------------------------------------------
# 6. Simple denoising idea via thresholding
# ------------------------------------------------

set.seed(1)
y <- f + rnorm(n, sd = 0.25)

dwt_y <- dwt(y, filter = "haar", n.levels = 4, boundary = "periodic")

# Hard threshold detail coefficients
thr <- 0.35
dwt_thr <- dwt_y
for (lev in 1:4) {
  dwt_thr@W[[lev]][abs(dwt_thr@W[[lev]]) < thr] <- 0
}

y_denoised <- idwt(dwt_thr)

plot(t, y, type = "l", lwd = 1,
     xlab = "t", ylab = "Signal",
     main = "Wavelet denoising")
lines(t, f, lwd = 2)
lines(t, y_denoised, lwd = 2, lty = 2)

legend("topright",
       legend = c("Noisy", "True", "Denoised"),
       lty = c(1, 1, 2), lwd = c(1, 2, 2), bty = "n")
```